

ALGORITHMIQUE ET PROGRAMMATION

Pr Amadou Dahirou GUEYE

Enseignant-chercheur en Informatique, UFR STA - UAM

Contact : dahirou.gueye@uam.edu.sn

Chapitre 4 : La programmation modulaire: Les sous programmes

1 Notions de procédure et fonction

1. Exemple 1

Supposons qu'on ait à écrire le programme permettant de calculer la fonction

$$f(x) = 2x^5 + 4x^3 + 1 \quad \text{pour un réel } x \text{ donné.}$$

S'il n'existe pas d'instruction permettant de coder le calcul de la puissance d'un nombre, l'algorithme va s'écrire comme suit :

Programme fonction ;

Variables x, f1, f2, f : réels i : entier ;

début

lire (x)

f1 := 1

pour i := 1 à 5 faire f1 := f1 * x

f2 := 1

pour i := 1 à 3 faire f2 := f2 * x

f := 2*f1 + 4*f2 + 1

ecrire ('f(', x, ') = ', f)

fin

On remarque que l'on utilise deux fois la même chose pour calculer les puissances de x (x^5 et x^3) et on pourrait paramétrer cela. Il suffira d'écrire un programme qui calcule la puissance d'un nombre et l'utiliser dans le programme précédent.

Programme puissance ;

Variables p, a : réels i, b : entier ;

début

lire (a, b)

$p := 1$

pour $i := 1$ à b faire

$p := p * a$

ecrire (a , ' puissance ', b , ' = ', p)

Fin

Pour calculer la valeur de **f1**, il suffit d'utiliser ce programme en donnant à **a** et **b** les valeurs de **x** et **5** à partir du premier programme (fonction). Cela signifie que les valeurs de **a** et **b** sont fournies non plus par saisie mais par le programme «utilisateur » c'est-à-dire fonction. Il nous faut donc réécrire le programme puissance sous la forme:

Programme puissance (a : réel ; b : entier)

Variables p : réel i : entier ;

début

p := 1

pour i := 1 à b faire

p := p * a

ecrire (a, ' puissance ', b ' = ', p)

fin

a et **b** sont les arguments du programme puissance.

Mais la valeur de **p** doit être récupérée dans le programme de calcul de la fonction dans la variable **f1** et ce n'est pas en l'affichant dans le programme puissance que cela va se faire. Il faut que le programme puissance retourne (ou renvoie) la valeur de **p** au programme

«utilisateur». Ainsi, à la place de l'instruction d'affichage

ecrire (a, ' puissance ', b ' = ', p)

on écrira **retourner (p)** ou **puissance := p**.

Ainsi écrit, le programme puissance pourra être utilisé par le premier programme comme suit :

Programme fonction ; Variables $x, f1, f2, f$: réels

début

lire (x)

$f1 := \text{puissance}(x, 5)$

$f2 := \text{puissance}(x, 3)$

$f := 2*f1 + 4*f2 + 1$

ecrire ($'f('x,') = ', f$)

fin

Quelles sont les particularités du programme puissance écrit et utilisé de cette façon ?

- (i) Il est écrit comme un programme (structurellement parlant).
- (iii) Il peut recevoir ses données d'entrée à partir d'un autre programme. Il peut renvoyer ses données de sortie à un autre programme.
- (iv) Son exécution est commandée par un autre programme.

Le programme puissance est un **sous programme**, plus précisément une **fonction**.

1.2 Définition

Un **sous programme** est rédigé de façon telle que son exécution puisse être commandée par un **programme**.

L'**exécution** d'un **sous programme** est donc déclenchée par un **programme**. Celui-ci est appelé **programme appelant** (ou **programme principal**). Il fournit des données au sous- programme et récupère les résultats de ce dernier.

En général, on distingue deux types de sous programme : les **procédures** et les **fonctions**.

La différence est qu'une fonction renvoie une valeur (c'est l'exemple du sous programme puissance) alors qu'une procédure ne renvoie pas de valeur.

En Pascal, le terme Programme est remplacé par le terme **function** s'il s'agit d'une fonction et **procedure** s'il s'agit d'une procédure. Généralement, les sous- programmes seront écrits à l'intérieur des programmes qui les utilisent, juste après la déclaration des variables.

Le programme Pascal complet de l'exemple est:

Program fonction ;

Var x, f1, f2, f: real;

(* definition du sous programme puissance *)

Function puissance (a :real ; b : integer) : real ;

Var p : real; i : integer;

begin

 p := 1 ;

 for i := 1 to b do

 p := p * a ;

 puissance := p ;

end ;

```
(* programme principal *)
```

```
begin
```

```
  readln(x) ;
```

```
  f1 := puissance (x,5) ;
```

```
  f2 := puissance (x,3) ;
```

```
  f := 2*f1 + 4*f2 + 1;
```

```
  writeln('f('x,') = ', f) ;
```

```
end.
```

1.3 Exemple 2

On veut écrire un programme Pascal qui saisit une heure et affiche «Bonjour» si on est avant 19h et «Bonsoir» sinon en utilisant deux procédures *SalutationsJour* et *SalutationsSoir* qui affichent respectivement «Bonjour» et «Bonsoir ».

```
Program salutations ;
```

```
var heure : integer ;
```

```
procedure SalutationsJour ;
```

```
begin
```

```
  writeln('Bonjour');
```

```
end ;
```



```
procedure SalutationsSoir ;
```

```
begin
```

```
    writeln('Bonsoir');
```

```
end ;
```

```
begin
```

```
    readln(heure);
```

```
    if heure < 19 then
```

```
        SalutationsJour
```

```
    else
```

```
        SalutationsSoir;
```

```
end.
```

4. Remarque

Quand un programme A utilise un sous programme B, il se passe successivement :

- a) Appel du sous programme B par le programme appelant A ;
- b) Suspension de l'exécution du programme A ;
- c) Exécution du sous programme B ;
- d) Retour au programme A.

2 Echange d'informations entre programme et sous-programme

1. Les variables globales et les variables locales

On distinguera les variables du programme principal, appelées **variables globales**, de celles des sous-programmes, appelées **variables locales**.

Les variables globales peuvent être utilisées aussi bien par le programme principal que par les sous programmes alors que les variables locales ne peuvent être utilisées que dans les sous programmes qui les ont définies (leur durée de vie est la durée d'exécution du sous programme).

Par exemple, considérons le programme suivant :

```
Program A ;  
var VA : integer ;  
Procedure B ;  
var VB : integer;  
begin  
    VB := 2*VA; writeln (VB) ;  
end;  
Procedure C ;  
var VC : integer;  
begin  
    readln (VC) ; VA := 3*VC;  
end;  
BEGIN (* programme principal A *)  
    C ; (* appel de la procédure C *)  
    B ; (* appel de la procédure B *)  
END.
```

Dans ce programme, VA est une variable globale ; elle peut être utilisée aussi bien dans le programme principal (A) que dans les procédures (B et C). VB et VC sont des variables locales aux procédures B et C ; elles ne peuvent pas être utilisées en dehors de ces procédures.

2.2 Les paramètres

2.2.1 Définition

Les paramètres permettent à un programme appelant de transmettre à un sous programme des données lors de l'appel de ce dernier.

Un sous programme est rédigé de façon à pouvoir recevoir des données du programme appelant : cela est possible grâce aux paramètres. On les appellera formels dans leur définition dans le sous programme et effectifs lors de l'appel du sous programme. A cet instant, les paramètres formels sont remplacés par des paramètres effectifs. Ces derniers sont fournis par le programme appelant.

2.2.2 Exemples

Essayons d'écrire un programme qui calcule et affiche la somme de deux entiers en utilisant :

a) une procédure sans paramètres

```
Program somme_a ;  
var x, y, s : integer ;  
Procedure som ;  
begin s := x + y; end;  
BEGIN  
  readln(x,y);  
  som; (* appel de la procédure som *)  
  writeln (s) ;  
END.
```

b) **une fonction sans paramètres**

```
Program somme_b ;
```

```
var x, y : integer ;
```

```
function som : integer ;
```

```
begin
```

```
    som := x + y;
```

```
end;
```

```
BEGIN
```

```
    readln(x,y);
```

```
    writeln (som) ; (* appel de la fonction som et affichage direct du résultat*)
```

```
END.
```

c) une procédure avec paramètres

```
Program somme_c ; var x, y, s : integer ;
```

```
Procedure som (a,b : integer);
```

```
begin
```

```
    s := a + b;
```

```
end;
```

```
BEGIN
```

```
    readln(x,y);
```

```
    som(x, y); (*appel de la procédure som *)
```

```
    writeln (s) ;
```

```
END.
```

d) une fonction avec paramètres

```
Program somme_d ;
```

```
var x, y : integer ;
```

```
function som (a,b:integer) : integer ;
```

```
begin
```

```
    som := a + b;
```

```
end;
```

```
BEGIN
```

```
    readln(x,y);
```

```
    writeln (som (x,y)) ; (* appel de la fonction som et affichage direct du résultat*)
```

```
END.
```

2.2.3 Le passage des paramètres

Rappelons que les variables globales existent pendant tout le temps que dure l'exécution du programme principal alors que les variables locales n'existent que durant l'exécution du sous programme qui les a déclarées. Ce qui fait que les valeurs affectées à ces variables sont perdues après l'exécution du sous programme.

Les paramètres des sous programmes se comportent comme des variables locales.

Exemple 1: Considérons le programme

suivant :

Program addition ;

```
var x, y, s : integer ;
```

```
function somme (a,b:integer) : integer ; begin
```

```
    somme := a + b;
```

```
end;
```

```
BEGIN
```

```
    readln(x,y);
```

```
    s := som (x,y) ;
```

```
    writeln ('la somme de ',x, 'et ',y,' vaut :',s);
```

```
END.
```

Dans ce programme, l'appel de la fonction *somme* avec les paramètres *x* et *y* donne les valeurs de ces deux variables à *a* et *b* qui sont créés à cet instant. Mais, leur existence ne dure que lors de l'exécution de la fonction.

Ici, on n'a pas besoin de garder *a* et *b* après l'exécution de la fonction *somme* mais il peut arriver qu'on ait besoin des valeurs prises par les paramètres d'un sous programme après son exécution.

Exemple 2 : Intéressons nous maintenant au programme suivant qui doit permettre d'échanger les valeurs contenues dans deux variables:

Program echange ;

var x, y : integer ;

procedure permuter (a,b:integer) ;

var c : integer;

begin

c := a;

a := b;

b := c;

end;

BEGIN

readln(x,y);

permuter (x,y) ;

writeln ('apres echange 'x,' vaut :', x, 'et 'y,' vaut :', y) ;

END.

Lors de l'appel de la procédure `permuter`, les valeurs des paramètres `a` et `b` sont permutées mais celles des variables `x` et `y` ne le sont pas.

Pour cela, il faudrait que lors du passage des paramètres effectifs (`x` et `y`), les paramètres formels (`a` et `b`) reçoivent les variables `x` et `y` et non leurs valeurs. On dira alors que les paramètres sont passés **par variable** ou **par adresse**, et non **par valeur** comme précédemment.

Il faut alors réécrire la procédure `permuter` comme suit :

```
procedure permuter (var a,b:integer) ; var c : integer;  
begin  
    c := a;  
    a := b;  
    b := c;  
end;
```

On a ainsi ajouté le terme **var** dans la déclaration des paramètres formels `a` et `b` de la procédure ; cela permet de signifier que le passage des paramètres se fait par variable. Si le terme **var** est omis à ce niveau devant un paramètre, cela veut dire que son passage se fait par valeur.

Exercice programme d'application:

Réécrire le permettant de calculer et d'afficher la somme de deux entiers en utilisant un passage par variable.

```
Program somme;
```

```
var x, y, s: integer;
```

```
procedure som (a,b: integer; var sp:integer);
```

```
begin
```

```
    sp:= a + b;
```

```
end;
```

```
BEGIN
```

```
readln(x,y);
```

```
som(x, y,s); (*appel de la procédure som *)
```

```
writeln(s);
```

```
END.
```

3. Exercices proposés

Exercice 1:

Ecrire un programme qui permet la saisie, l'inversion et l'affichage des éléments d'un tableau d'entiers. Le programme doit comprendre une procédure par opération.

Exercice 2:

- a) Ecrire une procédure permettant de déterminer si un nombre entier est premier. Elle comportera deux arguments: le nombre à examiner et un indicateur booléen précisant si ce nombre est premier ou non.
- b) Ecrire la procédure précédente sous forme d'une fonction.

Exercice 4:

Ecrire la procédure PASCAL *transfo_en_maj* (**minus**, **maj**) qui reçoit une chaîne de caractères minuscules (par le paramètre *minus*), et la transforme (dans le paramètre *maj*) en chaîne de caractères majuscules. On pourra utiliser la fonction prédéfinie *upcase* (*function upcase(c:char):char;*), qui reçoit en argument un caractère et retourne la majuscule correspondante.

Donner un exemple d'utilisation de la procédure *transfo_en_maj* à l'aide d'un petit programme PASCAL.

Exercice 5:

Ecrire un programme Pascal qui lit une chaîne de caractères et affiche le nombre de lettres de l'alphabetlatin et le nombre de chiffres contenus dans la chaîne. On peut utiliser les deux fonctions suivantes (*on ne vous demande pas de les écrire!*): *Function* **EstUneLettre** (*c:char*): *boolean* qui renvoie *true* si le paramètre reçu *c* est une lettre de l'alphabet latin et *false* sinon.

Function **EstUnChiffre** (*c:char*): *boolean* qui renvoie *true* si le paramètre reçu *c* est un chiffre et *false* sinon. Ecrire une procédure qui supprime tous les espaces d'une chaîne de caractères (de longueur max 50).

Exercice 6:

Ecrire une procédure DIVIS (**N**) qui met dans un ensemble tous les diviseurs naturels de l'entier strictement positif **N**.